

Technische Erklärung der Website

Für Rückfragen zu Aufbau, Code, Vorgehen, Prompts und behobenen Bugs

Kurzantwort für den Einstieg: Die Website ist eine statische Multi-Page-Anwendung aus HTML5, CSS und JavaScript. Sie besitzt kein Backend, keine Datenbank und kein Framework. HTML liefert Inhalt und Struktur, CSS das responsive Design und JavaScript die Interaktionen wie Navigation, Newsfeed, Quiz und datensparsame Medieneinbettung.

1. Architektur in vier Ebenen

Ebene	Aufgabe	Konkrete Dateien
HTML	Semantische Struktur, Texte, Links, Bilder und data-Attribute.	index.html, quiz.html, projektinfo.html und sechs Anwendungsseiten
CSS	Layout, Farben, Karten, Animationen, Responsive Design und Fokuszustände.	modern-ui.css, style.css, detail.css
JavaScript	Interaktive Zustände im Browser. Es werden keine Nutzerdaten an einen eigenen Server gesendet.	script.js, newsfeed.js, quiz.js, embed.js
Assets	Lokale Fotos und Anbieterlogos. Dadurch bleibt die Grunddarstellung auch offline erhalten.	assets/photos und assets/logos

2. Warum diese Architektur?

- Die Aufgabenstellung verlangt HTML, CSS und ein modernes Layout; dafür ist kein Framework nötig.
- Jede Anwendung besitzt eine eigene HTML-Seite. Das macht Navigation, Quellen und Präsentation übersichtlich.
- Gemeinsame Klassen sorgen dafür, dass alle Unterseiten gleich aussehen, obwohl ihre Inhalte unterschiedlich sind.
- Es gibt keinen Build-Schritt. index.html kann direkt lokal im Browser geöffnet werden.

3. So funktionieren die wichtigsten Komponenten

Komponente	Technische Funktionsweise
Navigation	script.js schaltet beim Menübutton die Klasse open und aria-expanded um. Beim Anklicken eines Links wird das mobile Menü wieder geschlossen.
Scrollfortschritt	Aus scrollY und der gesamten scrollbaren Höhe wird ein Prozentwert berechnet. Dieser Wert setzt die Breite von #progress-bar.
Scroll-Reveal	Ein IntersectionObserver beobachtet Elemente mit .reveal. Sobald 12 Prozent sichtbar sind, wird .visible ergänzt und das Element nicht weiter beobachtet.
Newsfeed	25 lokal definierte Einträge werden zufällig sortiert. requestAnimationFrame steuert einen 8-Sekunden-Timer und die Fortschrittsleiste. Vor und Zurück nutzen einen zyklischen Index.
Quiz	25 Begriffssätze werden mit vier Fragetypen zu genau 100 Fragen erweitert. Zehn Indizes werden zufällig gewählt und in sessionStorage mit der vorherigen Auswahl verglichen.
Externe Medien	embed.js sucht Karten mit data-provider und data-embed-url. Erst nach einem Klick wird ein iframe oder X-Skript erzeugt. Lade-, Fehler- und Fallback-Zustände bleiben sichtbar.

Vereinfachter Datenfluss

HTML-Element mit data-Attribut -> JavaScript liest den Zustand -> JavaScript ändert Text, Klasse oder Breite -> CSS stellt den neuen Zustand dar

Beispiel: Quizpool

25 Begriffe x 4 Fragetypen = 100 Fragen zufällige Auswahl = `shuffle(questionPool).slice(0, 10)`
Antwort = Buttons sperren + correct/wrong markieren Feedback = Überschrift + Erklärung + Alltagsbeispiel

4. Responsive Design und Accessibility

- Grid- und Flexbox-Layouts wechseln über Media Queries von drei oder zwei Spalten auf eine Spalte.
- CSS-Variablen speichern Farben, Abstände, Radien, Schatten und Animationszeiten zentral.
- focus-visible erzeugt klar erkennbare Tastaturrahmen; Skip-Link, Alt-Texte und ARIA-Beschriftungen helfen bei der Orientierung.
- prefers-reduced-motion reduziert Animationen für Personen, die weniger Bewegung eingestellt haben.
- Der Newsfeed meldet manuelle Wechsel über aria-live, und das Quiz meldet Erklärungen ebenfalls als Live-Region.

5. Verwendete Prompts - sinngemäß gekürzt

Wichtig: Die folgenden Formulierungen sind eine nachvollziehbare Zusammenfassung der Arbeitsaufträge, keine wortwörtliche Exportdatei des gesamten Chats. Sie zeigen, wie das Projekt schrittweise verfeinert wurde.

Phase	Beispiel für den Prompt	Ergebnis
Thema	Erstelle eine Website zum Thema KI im Alltag mit Chancen, Risiken und mehreren Anwendungen.	Grundidee, Startseite und erste Inhaltsstruktur
Unterseiten	Jede Anwendung soll eine eigene Unterseite mit grafischer Darstellung und Navigation erhalten.	Sechs Detailseiten mit gleicher Grundstruktur
Alltagsfokus	Erkläre weniger, wie KI technisch funktioniert, und stärker, wie sie im Alltag genutzt wird. Nutze reale Beispiele.	Texte wurden auf Lernen, Planung, Medizin, Mobilität und Kreativität ausgerichtet
Quellen	Ergänze offizielle Quellen, Medienbeispiele und einen datensparsamen Inhalt-laden-Mechanismus.	Quellenbereiche, lokale Vorschauen und embed.js
Bildung	Füge Chatbots in Schule und Studium mit Chancen und negativen Folgen ein.	Eigener Bildungsabschnitt mit KMK- und Hochschulquellen
Neue Beispiele	Erkläre ChatGPT Live im Full-Duplex-Modus und Neuralink als experimentelles medizinisches Beispiel.	Erweiterungen auf Sprach- und Medizinseite
Newsfeed	Erstelle einen breiten Feed mit Fortschrittsleiste, automatischem Wechsel und Pfeiltasten.	newsfeed.js mit lokaler Datenliste und Steuerung
Quiz	Nutze 100 Fragen, zehn pro Durchlauf und erkläre nach jeder Antwort, warum sie richtig ist.	quiz.js mit 25 Begriffen, vier Fragetypen und erklärender Sprechblase
Design	Überarbeite nur das visuelle Design: warmes Beige, Braun, Orange und Gold, hochwertig und responsive.	Zentrales Designsystem und bereinigte CSS-Struktur

6. Schrittweises Vorgehen

Schritt	Was gemacht wurde
1	Aufgabenstellung gelesen, Thema festgelegt und Zielgruppe eingeordnet.
2	Startseitenstruktur und Navigation mit semantischem HTML aufgebaut.
3	Sechs Anwendungsseiten erstellt und mit realen Alltagssituationen gefüllt.
4	Quellen, lokale Fotos, Logos, Medienkarten und Datenschutzinformationen ergänzt.
5	Newsfeed, Quiz, Navigation, Scroll-Reveal und Einbettungslogik mit JavaScript umgesetzt.
6	Design mehrfach überarbeitet und schließlich in ein zentrales warmes Designsystem überführt.
7	Kontraste, lokale Links, JavaScript-Syntax, Responsive Regeln und PDF-Präsentationsmaterial geprüft.

7. Aufgetretene Probleme und behobene Bugs

Problem	Ursache und Behebung
Uneinheitliche Farben	Alte Retro-, Neon- und helle Designregeln überlagerten sich. Farben, Schatten, Radian und Abstände wurden als CSS-Variablen in modern-ui.css zentralisiert.
Unschärfe Flip-Cards	3D-Transformationen machten die Rückseite unscharf. backface-visibility, translateZ, eine kontrollierte Perspektive und eine langsamere Drehung verbesserten die Darstellung.
Text passte nicht auf Karten	Rückseiten enthielten mehr Text als die Vorderseiten. Die Karte wurde beim Hover vergrößert, die Rückseite scrollbar gemacht und die Skalierung mobil deaktiviert.
Newsfeed-Leiste verschwand	Beim manuellen Wechsel lief der alte Animationszustand weiter. cancelAnimationFrame, zurückgesetzte Zeitwerte und ein zentraler startTimer-Aufruf beheben das Problem.
Quiz-Feedback war ungenau	Die Rückmeldung wiederholte nur die richtige Antwort. Definition, Begründung und Alltagsbeispiel wurden getrennte DOM-Elemente; falsche Antworten nennen zusätzlich die korrekte Lösung.
Maskottchen schlecht lesbar	Schrift und Hintergrund hatten zu wenig Kontrast. Größe, Schriftgewicht, Farben und die Verbindung zur Sprechblase wurden angepasst.
Externe Beiträge blockiert	Viele Seiten verhindern Einbettungen. Jede Medienkarte besitzt deshalb einen direkten Quellenlink und wechselt bei Fehlern in den Zustand is-error.
Datenschutz bei Medien	Ein sofortiges iframe hätte bereits beim Laden Drittanbieter kontaktiert. iframe oder X-Skript werden deshalb erst nach dem Klick auf Inhalt laden erzeugt.
Responsive Überläufe	Mehrspaltige Grids und lange Texte waren mobil zu breit. Media Queries stellen Karten, Flows, Quellen und Praxisbereiche schrittweise auf eine Spalte um.
Harte Kontraste	Reinweiß und sehr dunkle Flächen wirkten unruhig. Elfenbein, Beige, Taupe und Terrakotta wurden gewählt und wichtige Farbkombinationen auf mindestens 4,5 zu 1 geprüft.

Technische Grenzen, die man offen nennen kann

- Der Newsfeed ist kuratiert und lokal gespeichert. Er lädt keine Live-Nachrichten aus dem Internet.
- Die Website verwendet keine KI-API. Sie erklärt und demonstriert Anwendungsfälle, führt aber selbst kein KI-Modell aus.
- Externe Beiträge können durch Anbieterregeln oder fehlendes Internet blockiert sein; dafür gibt es Direktlinks.
- sessionStorage speichert nur die zuletzt gewählten Quiz-Indizes innerhalb der Browsersitzung, keine persönlichen Daten.

8. Typische technische Rückfragen und kurze Antworten

Frage	Kurze Antwort
Warum kein React oder Angular?	Für eine statische Schulwebsite wären sie unnötig komplex. HTML, CSS und JavaScript erfüllen die Anforderungen direkt und nachvollziehbar.
Ist das eine Single-Page-App?	Nein. Es ist eine Multi-Page-Website mit neun produktiven HTML-Seiten. Interne Links verbinden die Seiten und Abschnitte.
Woher kommen die 100 Quizfragen?	Aus 25 Begriffssätzen. Zu jedem Begriff erzeugt der Code vier unterschiedlich formulierte Fragetypen.
Wie wird verhindert, dass immer dieselben Fragen kommen?	Die Auswahl wird gemischt. Die vorherigen zehn Indizes liegen kurz in sessionStorage; bei zu großer Überschneidung wird neu gezogen.
Ist der Newsfeed live?	Nein. Er enthält 25 lokal gepflegte Einträge. Das vermeidet Tracking, Ausfälle und ungeprüfte Inhalte.
Warum werden Medien erst nach Klick geladen?	Damit beim ersten Seitenaufruf keine automatische Verbindung zu YouTube, X oder anderen Anbietern entsteht.
Wie funktioniert die Fortschrittsleiste?	requestAnimationFrame liefert Zeitstempel. Aus vergangener Zeit geteilt durch 8000 Millisekunden entsteht die prozentuale Breite.
Was passiert ohne JavaScript?	Texte, Bilder, Links und Seiten bleiben grundsätzlich lesbar. Interaktive Funktionen wie Feedwechsel, Quiz und Medienfreigabe benötigen JavaScript.
Wie wurde Responsivität erreicht?	Mit flexiblen Grids, clamp-Werten, relativen Breiten und Media Queries für Desktop, Tablet und Smartphone.
Welche Daten werden gespeichert?	Es gibt kein Konto, keine Datenbank und kein Tracking. Nur die Quiz-Auswahl wird vorübergehend in sessionStorage gespeichert.
Wie wurde Accessibility berücksichtigt?	Semantisches HTML, Alt-Texte, ARIA-Attribute, Skip-Link, Tastaturfokus, Live-Regionen und reduced-motion-Regeln.
Was würdet ihr technisch verbessern?	Früher ein Designsystem festlegen, automatisierte Browser-Tests ergänzen und Quellen sowie Newsfeed regelmäßig aktualisieren.

30-Sekunden-Schlussantwort

Technisch ist das Projekt bewusst nachvollziehbar aufgebaut: semantisches HTML für die Struktur, ein zentrales CSS-System für das responsive Design und kleine getrennte JavaScript-Dateien für klar abgegrenzte Funktionen. Die größten Herausforderungen waren konsistentes Design, mobile Darstellung, datensparsame Einbettungen, der Newsfeed-Timer und passende Quiz-Erklärungen. Diese Probleme wurden schrittweise getestet und behoben.

Nicht behaupten: Der Feed sei live, die Website nutze selbst eine KI-API oder Neuralink sei eine Heilung. Korrekt ist: Der Feed ist lokal kuratiert, die Seite erklärt KI-Anwendungen und Neuralink wird als experimentelle Studie dargestellt.